第一节：混合检索

混合检索（Hybrid Search）是一种结合了 **稀疏向量（Sparse Vectors）** 和 **密集向量（Dense Vectors）** 优势的先进搜索技术。旨在同时利用稀疏向量的关键词精确匹配能力和密集向量的语义理解能力，以克服单一向量检索的局限性，从而在各种搜索场景下提供更准确、更鲁棒的检索结果。

在本节中，我们将首先分析这两种核心向量的特性，然后讨论它们如何融合，最后通过milvus实现混合检索。

一、稀疏向量 vs 密集向量

为了更好地理解混合检索，首先需要厘清两种向量的本质区别。

1.1 稀疏向量

稀疏向量，也常被称为“词法向量”，是基于词频统计的传统信息检索方法的数学表示。它通常是一个维度极高（与词汇表大小相当）但绝大多数元素为零的向量。它采用精准的“词袋”匹配模型，将文档视为一堆词的集合，不考虑其顺序和语法，其中向量的每一个维度都直接对应一个具体的词，非零值则代表该词在文档中的重要性（权重）。这类向量的经典权重计算方法是 TF-IDF。在信息检索领域，BM25 则是基于这种稀疏表示的成功且应用广泛的排序算法之一，其核心公式如下：

$$ Score(Q, D) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})} $$

其中：

- $IDF(q_i)$: 查询词 q_i 的逆文档频率，用于衡量一个词的普遍程度。越常见的词，IDF值越低。
- $f(q_i, D)$: 查询词 q_i 在文档 D 中的词频。
- $|D|$: 文档 D 的长度。
- $avgdl$: 集合中所有文档的平均长度。
- k_1, b: 可调节的超参数。 k_1 用于控制词频饱和度（一个词在文档中出现10次和100次，其重要性增长并非线性）， b 用于控制文档长度归一化的程度。

这种方法的优点是可解释性极强（每个维度都代表一个确切的词），无需训练，能够实现关键词的精确匹配，对于专业术语和特定名词的检索效果好。主要缺点是无法理解语义，例如它无法识别“汽车”和“轿车”是同义词，存在“词汇鸿沟”。

1.2 密集向量

密集向量，也常被称为“语义向量”，是通过深度学习模型学习到的数据（如文本、图像）的低维、稠密的浮点数表示。这些向量旨在将原始数据映射到一个连续的、充满意义的“语义空间”中来捕捉“语义”或“概念”。在理想的语义空间中，向量之间的距离和方向代表了它们所表示概念之间的关系。一个经典的例子是 `vector('国王') - vector('男人') + vector('女人')` 的计算结果在向量空间中非常接近 `vector('女王')`，这表明模型学会了“性别”和“皇室”这两个维度的抽象概念。它的代表包括 Word2Vec、GloVe、以及所有基于 Transformer 的模型（如 BERT、GPT）生成的嵌入（Embeddings）。

其主要优点是能够理解同义词、近义词和上下文关系，泛化能力强，在语义搜索任务中表现卓越。但缺点也同样明显：可解释性差（向量中的每个维度通常没有具体的物理意义），需要大量数据和算力进行模型训练，且对于未登录词（OOV）[1] 的处理相对困难。

1.3 实例对比

稀疏向量表示:

稀疏向量的核心思想是只存储非零值。例如，一个8维的向量 `[0, 0, 0, 5, 0, 0, 0, 9]`，其大部分元素都是零。用稀疏格式表示，可以极大地节约空间。常见的稀疏表示法有两种：

1. **字典 / 键值对 (Dictionary / Key-Value):**
   这种方式将非零元素的 `索引` (0-based) 作为键，`值` 作为值。上面的向量可以表示为：

   ```
   // {索引: 值}
   {
     "3": 5,
     "7": 9
   }
   ```

2. **坐标列表 (Coordinate list - COO):**
   这种方式通常用一个元组 `(维度, [索引列表], [值列表])` 来表示。上面的向量可以表示为：

   ```
   (8, [3, 7], [5, 9])
   ```

   这种格式在 `SciPy` 等科学计算库中非常常见。

假设在一个包含5万个词的词汇表中，“西红柿”在第88位，“炒”在第666位，“蛋”在第999位，它们的BM25权重分别是1.2、0.8、1.5。那么它的稀疏表示（采用字典格式）就是：

```
// {索引: 权重}
{
  "88": 1.2,
  "666": 0.8,
  "999": 1.5
}
```

如果采用坐标列表（COO）格式，它会是这样：

```
(50000, [88, 666, 999], [1.2, 0.8, 1.5])
```

这两种格式都清晰地记录了文档的关键信息，但它们的局限性也很明显：如果我们搜索“番茄炒鸡蛋”，由于“番茄”和“西红柿”是不同的词条（索引不同），模型将无法理解它们的语义相似性。

密集向量表示:

与稀疏向量不同，密集向量的所有维度都有值，因此使用**数组** `[]` 来表示是最直接的方式。一个预训练好的语义模型在读取“西红柿炒蛋”后，会输出一个低维的密集向量：

```
// 这是一个低维（比如1024维）的浮点数向量
// 向量的每个维度没有直接的、可解释的含义
[0.89, -0.12, 0.77, ..., -0.45]
```

这个向量本身难以解读，但它在语义空间中的位置可能与“番茄鸡蛋面”、“洋葱炒鸡蛋”等菜肴的向量非常接近，因为模型理解了它们共享“鸡蛋类菜肴”、“家常菜”、“酸甜口味”等核心概念。因此，当我们搜索“蛋白质丰富的家常菜”时，即使查询中没有出现任何原文关键词，密集向量也很有可能成功匹配到这份菜谱。

二、混合检索

通过上文可以看出稀疏向量和密集向量各有千秋，那么将它们结合起来，实现优势互补，就成了一个不错的选择。混合检索便是基于这个思路，通过结合多种搜索算法（最常见的是稀疏与密集检索）来提升搜索结果相关性和召回率。

- **主要目标**：解决单一检索技术的局限性。例如，关键词检索无法理解语义，而向量检索则可能忽略掉必须精确匹配的关键词（如产品型号、函数名等）。混合检索旨在同时利用稀疏向量的**精确性**和密集向量的**泛化性**，以应对复杂多变的搜索需求。

2.1 技术原理与融合方法

混合检索通常并行执行两种检索算法，然后将两组异构的结果集融合成一个统一的排序列表。以下是两种主流的融合策略：

2.1.1 倒数排序融合 (Reciprocal Rank Fusion, RRF)

RRF 不关心不同检索系统的原始得分，只关心每个文档在各自结果集中的**排名**。其思想是：一个文档在不同检索系统中的排名越靠前，它的最终得分就越高。

其计分公式为：

$$ RRF*{score}(d) = \sum*{i=1}^{k} \frac{1}{rank_i(d) + c} $$

其中：

- d 是待评分的文档。
- k 是检索系统的数量（这里是2，即稀疏和密集）。
- $rank_i(d)$ 是文档 d 在第 i 个检索系统中的排名。
- c 是一个常数（通常设为60），用于降低排名靠后文档的权重，避免它们对结果产生过大影响。

2.1.2 加权线性组合

这种方法需要先将不同检索系统的得分进行归一化（例如，统一到 0-1 区间），然后通过一个权重参数 `α` 来进行线性组合。

$$ Hybrid*{score} = \alpha \cdot Dense*{score} + (1 - \alpha) \cdot Sparse_{score} $$

通过调整 `α` 的值，可以灵活地控制语义相似性与关键词匹配在最终排序中的贡献比例。例如，在电商搜索中，可以调高关键词的权重；而在智能问答中，则可以侧重于语义。

2.2 优势与局限

优势	局限
召回率与准确率高：能同时捕获关键词和语义，显著优于单一检索。	**计算资源消耗大**：需要同时维护和查询两套索引。
灵活性强：可通过融合策略和权重调整，适应不同业务场景。	**参数调试复杂**：融合权重等超参数需要反复实验调优。
容错性好：关键词检索可部分弥补向量模型对拼写错误或罕见词的敏感性。	**可解释性仍是挑战**：融合后的结果排序理由难以直观分析。

三、代码实践：通过 Milvus 实现混合检索

接下来使用 Milvus 来实现一个完整的混合检索流程，从定义 Schema、插入数据，到执行查询。

3.1 步骤一：定义 Collection

在上一章中我们实现了多模态图文检索，现在还是同样的步骤先创建一个 Collection。

```
import json
import os
os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
import numpy as np
from pymilvus import connections, MilvusClient, FieldSchema, CollectionSchema,
DataType, Collection, AnnSearchRequest, RRFRanker
from pymilvus.model.hybrid import BGEM3EmbeddingFunction

# 1. 初始化设置
COLLECTION_NAME = "dragon_hybrid_demo"
MILVUS_URI = "http://localhost:19530"  # 服务器模式
DATA_PATH = "../../data/C4/metadata/dragon.json"  # 相对路径
BATCH_SIZE = 50

# 2. 连接 Milvus 并初始化嵌入模型
print(f"--> 正在连接到 Milvus: {MILVUS_URI}")
connections.connect(uri=MILVUS_URI)

print("--> 正在初始化 BGE-M3 嵌入模型...")
ef = BGEM3EmbeddingFunction(use_fp16=False, device="cpu")
print(f"--> 嵌入模型初始化完成。密集向量维度: {ef.dim['dense']}")

# 3. 创建 Collection
milvus_client = MilvusClient(uri=MILVUS_URI)
if milvus_client.has_collection(COLLECTION_NAME):
    print(f"--> 正在删除已存在的 Collection '{COLLECTION_NAME}'...")
    milvus_client.drop_collection(COLLECTION_NAME)

fields = [
    FieldSchema(name="pk", dtype=DataType.VARCHAR, is_primary=True,
auto_id=True, max_length=100),
    FieldSchema(name="img_id", dtype=DataType.VARCHAR, max_length=100),
    FieldSchema(name="path", dtype=DataType.VARCHAR, max_length=256),
    FieldSchema(name="title", dtype=DataType.VARCHAR, max_length=256),
    FieldSchema(name="description", dtype=DataType.VARCHAR, max_length=4096),
    FieldSchema(name="category", dtype=DataType.VARCHAR, max_length=64),
    FieldSchema(name="location", dtype=DataType.VARCHAR, max_length=128),
    FieldSchema(name="environment", dtype=DataType.VARCHAR, max_length=64),
    FieldSchema(name="sparse_vector", dtype=DataType.SPARSE_FLOAT_VECTOR),
    FieldSchema(name="dense_vector", dtype=DataType.FLOAT_VECTOR,
dim=ef.dim["dense"])
]

# 如果集合不存在，则创建它及索引
if not milvus_client.has_collection(COLLECTION_NAME):
    print(f"--> 正在创建 Collection '{COLLECTION_NAME}'...")
    schema = CollectionSchema(fields, description="关于龙的混合检索示例")
    # 创建集合
```

```
    collection = Collection(name=COLLECTION_NAME, schema=schema,
consistency_level="Strong")
    print("--> Collection 创建成功。")

    # 创建索引
    print("--> 正在为新集合创建索引...")
    sparse_index = {"index_type": "SPARSE_INVERTED_INDEX", "metric_type": "IP"}
    collection.create_index("sparse_vector", sparse_index)
    print("稀疏向量索引创建成功。")

    dense_index = {"index_type": "AUTOINDEX", "metric_type": "IP"}
    collection.create_index("dense_vector", dense_index)
    print("密集向量索引创建成功。")

collection = Collection(COLLECTION_NAME)
collection.load()
print(f"--> Collection '{COLLECTION_NAME}' 已加载到内存。")
```

fields字段类型分析:

- **pk**: 主键设计，`auto_id=True` 让 Milvus 自动生成唯一标识，避免主键冲突
- **标量字段**: 7个VARCHAR字段用于存储元数据，`max_length` 根据实际数据分布优化存储
- **稀疏向量**: `SPARSE_FLOAT_VECTOR` 类型，存储关键词权重
- **密集向量**: `FLOAT_VECTOR` 类型，固定1024维，存储语义特征

3.2 步骤二：BGE-M3 双向量生成

这里使用 BGE-M3 作为向量生成器，它能够同时生成稀疏向量和密集向量。

3.2.1 数据加载与预处理

```
if collection.is_empty:
    print(f"--> Collection 为空，开始插入数据...")
    with open(DATA_PATH, 'r', encoding='utf-8') as f:
        dataset = json.load(f)

    docs, metadata = [], []
    for item in dataset:
        parts = [
            item.get('title', ''),
            item.get('description', ''),
            item.get('location', ''),
            item.get('environment', ''),
        ]
        docs.append(' '.join(filter(None, parts)))
        metadata.append(item)
```

Collection 此时已加载到内存但为空状态。通过 `is_empty` 检查避免重复插入。多字段文本合并中每个实体对应一个完整的数据记录。

3.2.2 向量生成

```
print("--> 正在生成向量嵌入...")
embeddings = ef(docs)
print("--> 向量生成完成。")

# 获取两种向量
sparse_vectors = embeddings["sparse"]     # 稀疏向量：词频统计
dense_vectors = embeddings["dense"]       # 密集向量：语义编码
```

3.2.3 Collection 批量数据插入

```
# 为每个字段准备批量数据
img_ids = [doc["img_id"] for doc in metadata]
paths = [doc["path"] for doc in metadata]
titles = [doc["title"] for doc in metadata]
descriptions = [doc["description"] for doc in metadata]
categories = [doc["category"] for doc in metadata]
locations = [doc["location"] for doc in metadata]
environments = [doc["environment"] for doc in metadata]

# 插入数据
collection.insert([
    img_ids, paths, titles, descriptions, categories, locations, environments,
    sparse_vectors, dense_vectors
])
collection.flush()
```

- **字段映射**: 严格按照 Schema 定义的字段顺序插入，9个字段（7个标量+2个向量）
- `flush()` **作用**: 强制将内存缓冲区数据写入磁盘，使数据立即可搜索
- **最终状态**: Collection 包含6个Entity，索引层使用稀疏向量的 `SPARSE_INVERTED_INDEX` 和密集向量的 `AUTOINDEX`

3.3 步骤三：实现混合检索

最后使用 milvus 中封装好的 RRF 排序算法来完成混合检索：

3.3.1 查询向量生成

```
# 6. 执行搜索
search_query = "悬崖上的巨龙"
search_filter = 'category in ["western_dragon", "chinese_dragon",
"movie_character"]'
top_k = 5

print(f"\n{'='*20} 开始混合搜索 {'='*20}")
print(f"查询: '{search_query}'")
print(f"过滤器: '{search_filter}'")

# 生成查询向量
query_embeddings = ef([search_query])
dense_vec = query_embeddings["dense"][0]
sparse_vec = query_embeddings["sparse"]._getrow(0)
```

尝试打印向量信息可以看到如下输出：

```
=== 向量信息 ===
密集向量维度: 1024
密集向量前5个元素: [-0.0035305   0.02043397 -0.04192593 -0.03036701 -0.02098157]
密集向量范数: 1.0000

稀疏向量维度: 250002
稀疏向量非零元素数量: 6
稀疏向量前5个非零元素:
  - 索引: 6, 值: 0.0659
  - 索引: 7977, 值: 0.1459
  - 索引: 14732, 值: 0.2959
  - 索引: 31433, 值: 0.1463
  - 索引: 141121, 值: 0.1587

稀疏向量密度: 0.00239998%
```

3.3.2 混合检索执行

使用 RRF 算法进行混合检索，通过 milvus 封装的 RRFRanker 实现。RRFRanker 的核心参数是 `k` 值（默认60），用于控制 RRF 算法中的排序平滑程度。

其中 `k` 值越大，排序结果越平滑；越小则高排名结果的权重越突出

```
# 定义搜索参数
search_params = {"metric_type": "IP", "params": {}}

# 先执行单独的搜索
print("\n--- [单独] 密集向量搜索结果 ---")
dense_results = collection.search(
    [dense_vec],
    anns_field="dense_vector",
    param=search_params,
    limit=top_k,
    expr=search_filter,
    output_fields=["title", "path", "description", "category", "location",
"environment"]
)[0]

for i, hit in enumerate(dense_results):
    print(f"{i+1}. {hit.entity.get('title')} (Score: {hit.distance:.4f})")
    print(f"    路径: {hit.entity.get('path')}")
    print(f"    描述: {hit.entity.get('description')[:100]}...")

print("\n--- [单独] 稀疏向量搜索结果 ---")
sparse_results = collection.search(
    [sparse_vec],
    anns_field="sparse_vector",
    param=search_params,
    limit=top_k,
    expr=search_filter,
    output_fields=["title", "path", "description", "category", "location",
"environment"]
)[0]
```

```
for i, hit in enumerate(sparse_results):
    print(f"{i+1}. {hit.entity.get('title')} (Score: {hit.distance:.4f})")
    print(f"    路径: {hit.entity.get('path')}")
    print(f"    描述: {hit.entity.get('description')[:100]}...")

print("\n--- [混合] 稀疏+密集向量搜索结果 ---")
# 创建 RRF 融合器
rerank = RRFRanker(k=60)

# 创建搜索请求
dense_req = AnnSearchRequest([dense_vec], "dense_vector", search_params,
limit=top_k)
sparse_req = AnnSearchRequest([sparse_vec], "sparse_vector", search_params,
limit=top_k)

# 执行混合搜索
results = collection.hybrid_search(
    [sparse_req, dense_req],
    rerank=rerank,
    limit=top_k,
    output_fields=["title", "path", "description", "category", "location",
"environment"]
)[0]

# 打印最终结果
for i, hit in enumerate(results):
    print(f"{i+1}. {hit.entity.get('title')} (Score: {hit.distance:.4f})")
    print(f"    路径: {hit.entity.get('path')}")
    print(f"    描述: {hit.entity.get('description')[:100]}...")
```

最终输出如下:

```
--- [单独] 密集向量搜索结果 ---
1. 悬崖上的白龙 (Score: 0.7219)
    路径: ../../data/C3/dragon/dragon02.png
    描述: 一头雄伟的白色巨龙栖息在悬崖边缘，背景是金色的云霞和远方的海岸。它拥有巨大的翅膀和优雅
的身姿，是典型的西方奇幻生物。...
2. 中华金龙 (Score: 0.5131)
    路径: ../../data/C3/dragon/dragon06.png
    描述: 一条金色的中华龙在祥云间盘旋，它身形矫健，龙须飘逸，展现了东方神话中龙的威严与神
圣。...
3. 驯龙高手：无牙仔 (Score: 0.5119)
    路径: ../../data/C3/dragon/dragon05.png
    描述: 在电影《驯龙高手》中，主角小嗝嗝骑着他的龙伙伴无牙仔在高空飞翔。他们飞向灿烂的太阳，下
方是岛屿和海洋，画面充满了冒险与友谊。...

--- [单独] 稀疏向量搜索结果 ---
1. 悬崖上的白龙 (Score: 0.2319)
    路径: ../../data/C3/dragon/dragon02.png
    描述: 一头雄伟的白色巨龙栖息在悬崖边缘，背景是金色的云霞和远方的海岸。它拥有巨大的翅膀和优雅
的身姿，是典型的西方奇幻生物。...
2. 中华金龙 (Score: 0.0923)
    路径: ../../data/C3/dragon/dragon06.png
```

```
    描述: 一条金色的中华龙在祥云间盘旋，它身形矫健，龙须飘逸，展现了东方神话中龙的威严与神
圣。...
3. 驯龙高手：无牙仔 (Score: 0.0691)
    路径: ../../data/C3/dragon/dragon05.png
    描述: 在电影《驯龙高手》中，主角小嗝嗝骑着他的龙伙伴无牙仔在高空飞翔。他们飞向灿烂的太阳，下
方是岛屿和海洋，画面充满了冒险与友谊。...

--- [混合] 稀疏+密集向量搜索结果 ---
1. 悬崖上的白龙 (Score: 0.0328)
    路径: ../../data/C3/dragon/dragon02.png
    描述: 一头雄伟的白色巨龙栖息在悬崖边缘，背景是金色的云霞和远方的海岸。它拥有巨大的翅膀和优雅
的身姿，是典型的西方奇幻生物。...
2. 中华金龙 (Score: 0.0320)
    路径: ../../data/C3/dragon/dragon06.png
    描述: 一条金色的中华龙在祥云间盘旋，它身形矫健，龙须飘逸，展现了东方神话中龙的威严与神
圣。...
3. 霸王龙的怒吼 (Score: 0.0318)
    路径: ../../data/C3/dragon/dragon03.png
    描述: 史前时代的霸王龙张开血盆大口，发出震天的怒吼。在它身后，几只翼龙在阴沉的天空中盘旋，展
现了白垩纪的原始力量。...
4. 奔跑的奶龙 (Score: 0.0313)
    路径: ../../data/C3/dragon/dragon04.png
    描述: 一只Q版的黄色小恐龙，有着大大的绿色眼睛和友善的微笑。是一部动画中的角色，非常可
爱。...
5. 驯龙高手：无牙仔 (Score: 0.0310)
    路径: ../../data/C3/dragon/dragon05.png
    描述: 在电影《驯龙高手》中，主角小嗝嗝骑着他的龙伙伴无牙仔在高空飞翔。他们飞向灿烂的太阳，下
方是岛屿和海洋，画面充满了冒险与友谊。...
```

> 本节完整代码

练习

- 分析算法为什么在密集向量检索和稀疏向量检索中，排名第三的驯龙高手在混合检索中反而排在了第五?
- 基于上一节的多模态检索代码 `04_multi_milvus.py` ，结合本节的检索代码加入多模态信息融合的功能并尝试使用混合检索。 (参考代码)

脚注

1. **OOV（Out-of-Vocabulary）未登录词**：指在模型训练时没有出现在词汇表中，但在实际使用时遇到的新词汇。例如，如果模型训练时词汇表中没有"ChatGPT"这个词，那么在实际应用中遇到它时就是OOV。传统的稀疏向量方法（如BM25）对OOV词汇会完全忽略，而现代的密集向量方法通过子词分割（如BPE、WordPiece）可以更好地处理OOV问题。 ↩